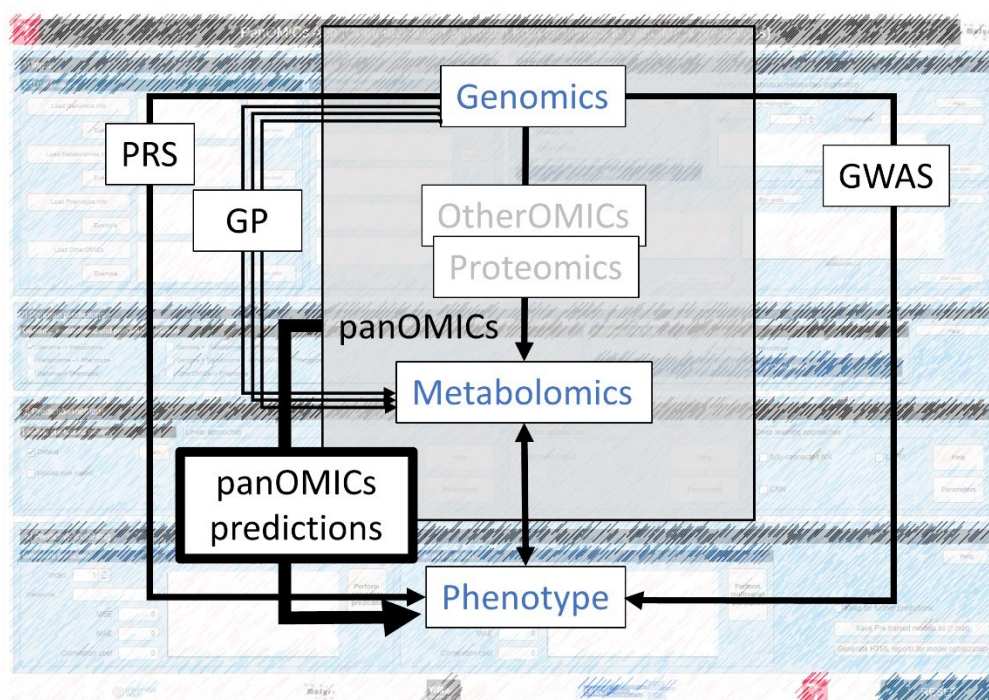




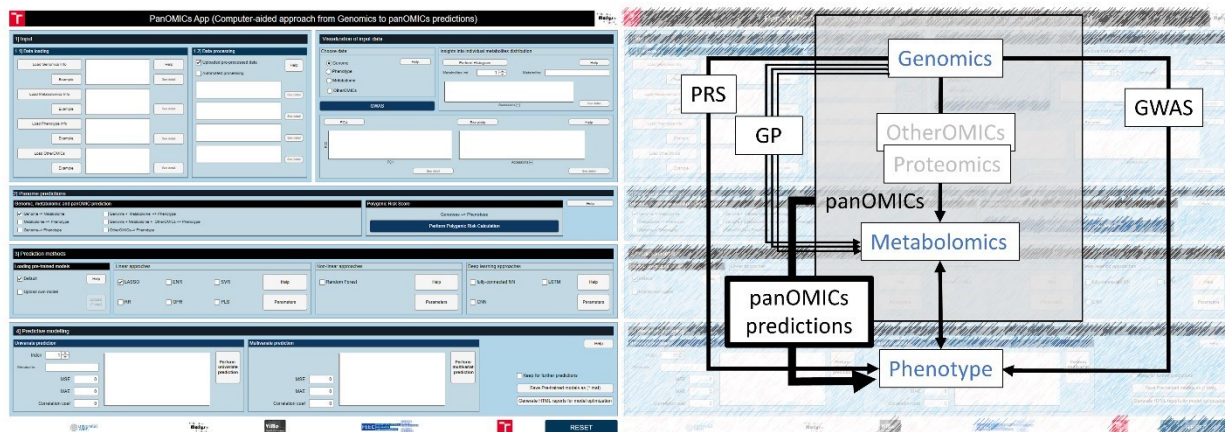
PANOMICS App Manual



Introduction

Advancements in sequencing technologies have propelled molecular biology into the post-genomic era, where research increasingly focuses on understanding the functional relationships between genetic variation and phenotypic outcomes.

The PANOMICS App, developed in MATLAB R2023b App Designer, provides an integrated and user-friendly environment for uploading, processing, visualizing, and analyzing genomic and metabolomic data. The application supports multiple prediction frameworks, including genomic, polygenic, and panOMICS approaches, and implements a range of linear, non-linear, and deep learning methods.



Installation

Option 1: MATLAB installed

If MATLAB is installed on your system, the application can be executed directly:

1. Navigate to:

PANOMICSApp/for_testing/

2. Run:

PANOMICs.exe

Option 2: Standalone version (no MATLAB required)

If MATLAB is not available or licensed:

1. Navigate to:

PANOMICSApp/for_redistribution/

2. Run the installer:

MyAppInstaller_web.exe

3. After installation, download the example input data from:

PANOMICSApp/for_testing/input_example

4. Place the data in the same directory as the installed application.

Important (required for application launch):

Before running the standalone application, it is necessary to download and install the MATLAB Runtime R2023b (23.2).

The runtime is available for free and no MATLAB license is required to run the full desktop application.

Download here:

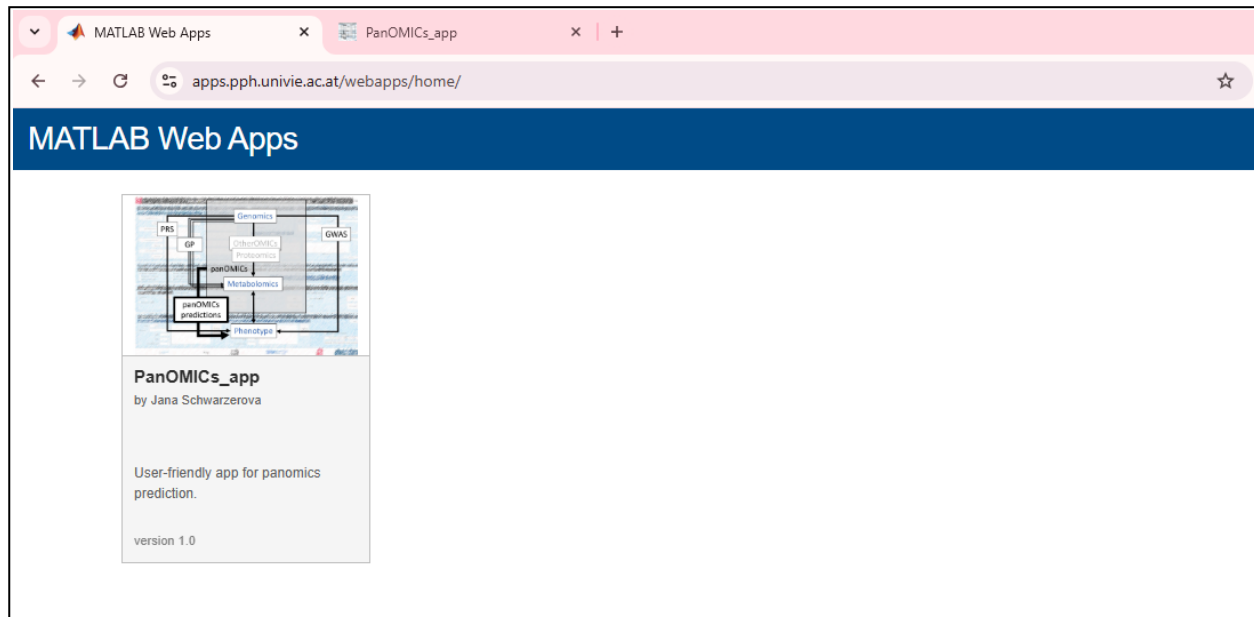
<https://www.mathworks.com/products/compiler/matlab-runtime.html>

Install the runtime first, then proceed with the application installation.

Option 3: Web-based version

A web-based version of the PANOMICS App is also available for demonstration and lightweight analyses:

<https://apps.pph.univie.ac.at/webapps/home/>



The web and desktop versions share a consistent interface and do not require a MATLAB license. Users can upload datasets, perform preprocessing, visualize data, and run prediction analyses directly through the browser.

The web version provides the full core functionality of the application; however, performance may be limited by server-side computational resources, particularly for large datasets or computationally intensive analyses. For such cases, the standalone desktop version is recommended.

Supported Operating Systems

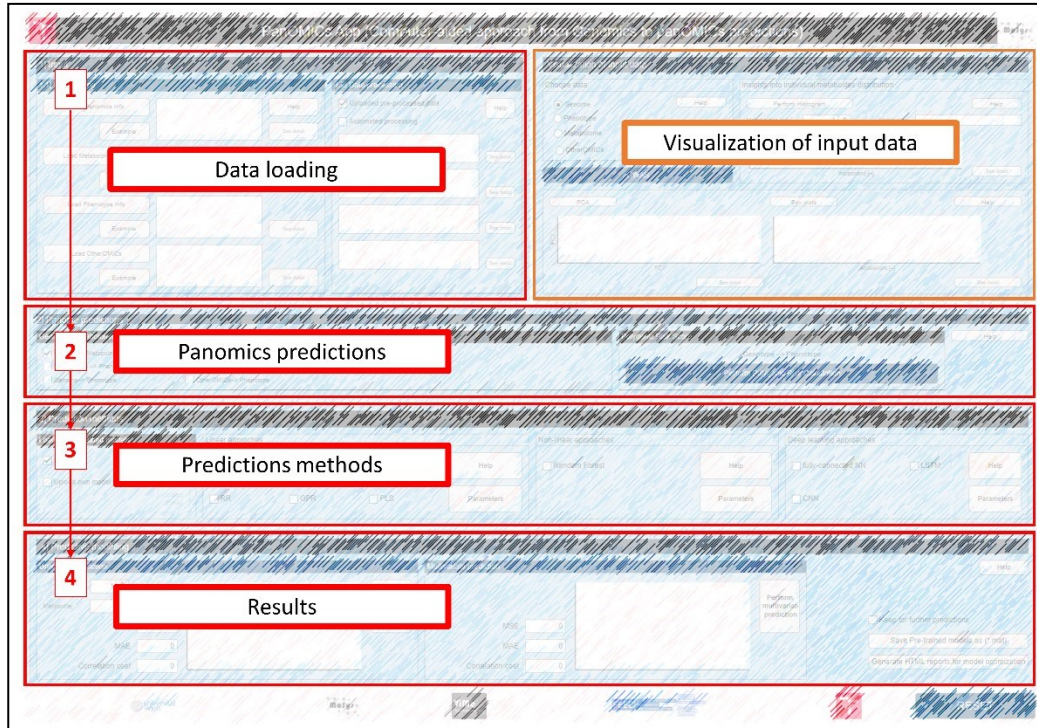
The PANOMICS App is available as a standalone desktop application for Windows and Linux. The installation instructions above describe the Windows distribution.

For installation on Linux or information regarding deployment on macOS, please contact **Jana.Schwarzerova@vut.cz**. Platform-specific installation instructions and information about compatible application builds will be provided upon request.

The standalone application requires a compatible MATLAB Runtime installation but does not require a commercial MATLAB license.

Application Workflow and Functionality

The PANOMICS App is organized into four main functional sections that guide users through data upload, exploration, prediction, and output management.



PanOMICS App (Computer-aided approach from Genomics to panOMICS predictions)

1) Input

1.1) Data loading

Load Genomics Info [Example] [Help] [See detail]

Load Metabolomics Info [Example] [See detail]

Load Phenotype Info [Example] [See detail]

Load Other OMICS [Example] [See detail]

1.2) Data processing

☒ Uploaded pre-processed data [Help]

☐ Automated processing [See detail]

Visualization of input data

Choose data: ☒ Genotype [Help] ☐ Phenotype ☐ Metabolome ☐ Other OMICS

Perform Histogram [Help]

Metabolites ind. [1-2] Metabolite [See detail]

2) Panomics predictions

Genomic, metabolomic and panOMICS prediction

☒ Genotype → Metabolome ☐ Genotype → Metabolome → Phenotype

☐ Metabolome → Phenotype ☐ Genotype → Metabolome → Other OMICS → Phenotype

☐ Other OMICS → Phenotype

Polygenic Risk Score [Help]

Genotype → Phenotype

Perform Polygenic Risk Calculation

3) Prediction methods

Loading pre-trained models

☒ Default [Help] ☐ Upload own model [Upload (.mat)]

Linear approaches

☒ LASSO ☐ ENR ☐ SVR [Help]

☐ RR ☐ GPR ☐ PLS [Parameters]

Non-linear approaches

☐ Random Forest [Help] [Parameters]

Deep learning approaches

☐ Fully-connected NN ☐ LSTM [Help]

☐ CNN [Parameters]

4) Predictive modelling

Univariate prediction

Index [1-2]

Metabolite [See detail]

MSE [0] MAE [0] Correlation coef [0]

Perform univariate prediction

Multivariate prediction

MSE [0] MAE [0] Correlation coef [0]

Perform multivariate prediction

☐ Keep for further predictions

Save Pre-trained models as (.mat)

Generate HTML reports for model optimization

RESET

1) Input section

Input section of the app is designed for uploading data. The app expects genomic data in the form of SNP matrices, where '0' indicates an undetected SNP and '1' indicates a detected SNP. Users can also upload metabolomic data related to individual samples associated with the SNP data. Additionally, phenotypic data can be uploaded for calculating Polygenic Risk Scores (PRS) and panOMICS predictions.

The screenshot displays the '1) Input' section of the application interface. It is divided into two main panels: '1.1] Data loading' and '1.2] Data processing'.

1.1] Data loading panel contains four rows of input fields:

- Load Genomics Info:** Includes a 'Load Genomics Info' button, an 'Example' button, a large text input area, a 'Help' button, and a 'See detail' button.
- Load Metabolomics Info:** Includes a 'Load Metabolomics Info' button, an 'Example' button, a large text input area, a 'See detail' button, and a 'See detail' button.
- Load Phenotype Info:** Includes a 'Load Phenotype Info' button, an 'Example' button, a large text input area, a 'See detail' button, and a 'See detail' button.
- Load OtherOMICS:** Includes a 'Load OtherOMICS' button, an 'Example' button, a large text input area, and a 'See detail' button.

1.2] Data processing panel contains:

- Two radio buttons: 'Uploaded pre-processed data' (checked) and 'Automated processing' (unchecked). Each has a 'Help' button.
- Four large text input areas, each with a 'See detail' button.

Data Preprocessing

Users can upload genomic, metabolomic, and phenotypic datasets and select either previously processed data or the automatic preprocessing option.

For genomic predictors, missing values are replaced with zeros according to the implemented preprocessing strategy. For metabolomic data, missing measurements are replaced with half of the minimum detected value, while observed measurements are preserved. A log10 transformation is subsequently applied to metabolite intensities.

Importantly, missing phenotypic values are not automatically replaced with zeros. Instead, missing target values are preserved, and users are informed that samples with missing outcomes must be excluded before supervised model training.

Important methodological consideration: Data-dependent preprocessing operations performed on the complete dataset before training and testing may introduce information leakage. For rigorous predictive evaluation, preprocessing parameters should be estimated exclusively from training data and subsequently applied to held-out observations.



1] Input

1.1] Data loading

Load Genomics Info

Load Metabolomics Info

Load Phenotype Info

Load Other OMICs

1.2] Data processing

☒ Uploaded pre-processed data

Visualization of input data

Choose data: ☐ Genome

2] Panome predictions

Genomic, metabolomic and panOMIC prediction

☒ Genome -> Metabolome ☐ Genom...

☐ Metabolome -> Phenotype ☐ Genom...

☐ Genome -> Phenotype ☐ Other O...

3] Prediction methods

Loading pre-trained models

Load a dataset

« _Final_PanOMICS_App_Bioinformatis_submission > App > input_example > input_G_M_Phe

Search input_G_M_Phe

Name	Date modified	Type	Size
Genotype_Ath.csv	09/08/2024 10:29	Microsoft Excel C...	121 KB
Metabolites_Ath_6C.csv	30/07/2024 11:40	Microsoft Excel C...	110 KB
Metabolites_Ath_16C.csv	30/07/2024 11:45	Microsoft Excel C...	110 KB
Phenotype_Ath_RGR_6.csv	30/07/2024 11:43	Microsoft Excel C...	5 KB
Phenotype_Ath_RGR_16.csv	29/07/2024 14:08	Microsoft Excel C...	5 KB

File name: Genotype_Ath.csv

All Files (*.*)

1

2

3

Visualization section

The Visualization module enables interactive exploration of input datasets. Available tools include principal component analysis (PCA), boxplots, and histograms for both genomic and metabolomic data distributions. In addition, genome-wide association study (GWAS) analysis can be performed for genomic datasets, providing a comprehensive overview of underlying genetic structure and variability.



2| Panome prediction section

The central functionality of the application is implemented in the Panome prediction section, where users define the overall prediction framework. This includes the selection of the prediction task, such as genomic prediction, metabolome prediction, other-omics integration, or polygenic risk score (PRS) estimation. The section provides a structured interface that links data input with downstream modelling and enables seamless transition to the prediction algorithms implemented in the Prediction Methods section.

The screenshot displays the '2| Panome predictions' section of the application. It features a top navigation bar with a 'Help' button. Below the bar, there are two main panels. The left panel, titled 'Genomic, metabolomic and panOMIC prediction', contains several checkboxes for selecting the prediction task: 'Genome -> Metabolome' (checked), 'Metabolome -> Phenotype', 'Genome -> Phenotype', 'Genome + Metabolome -> Phenotype', 'Genome + Metabolome + OtherOMiCs -> Phenotype', and 'OtherOMiCs -> Phenotype'. The right panel, titled 'Polygenic Risk Score', shows a 'Genotype -> Phenotype' workflow with a 'Perform Polygenic Risk Calculation' button. A blue arrow points from this button to the 'PRS (Polygenic Risk Score)' section below. This section includes a 'Load control group' and 'Load case group' area with 'Example data' lists. Below these is a 'Calculate PRS' button. The bottom part of the section features a histogram titled 'PRS (Control/Case) distribution' showing the distribution of PRS values for controls (red) and cases (blue). To the right of the histogram is a table with columns for 'SNP', 'beta', 'p-value', and 'beta' (repeated). The table lists various SNPs and their associated beta coefficients and p-values. At the bottom, there are buttons for 'Download results control group', 'Download results case group', and 'Back to MAIN'.

Polygenic Risk Score (PRS) Calculation

The PRS workflow separates SNP effect-size estimation from score evaluation. Genotype and phenotype data are divided into a discovery subset (70%) and an independent target subset (30%).

SNP effect sizes (beta coefficients) are estimated using univariate regression within the discovery subset and subsequently applied to individuals in the held-out target subset. The same estimated SNP effects are used for both case and control individuals.

In the current implementation, SNP-associated p-values are calculated but are not used for

SNP filtering. Linkage disequilibrium (LD) clumping is not performed.

For binary outcomes, predictive discrimination is evaluated using the area under the receiver operating characteristic curve (AUC) in the held-out target subset.

The PRS module is intended primarily for exploratory research and methodological demonstration rather than clinically validated risk prediction.

3| Prediction methods section

The screenshot shows a web interface titled "3| Prediction methods". It is divided into four main sections: "Loading pre-trained models", "Linear approaches", "Non-linear approaches", and "Deep learning approaches".

- Loading pre-trained models:** Contains a checked "Default" checkbox with a "Help" button, an unchecked "Upload own model" checkbox, and an "Upload (*.mat)" button.
- Linear approaches:** Contains checkboxes for LASSO (checked), ENR, SVR, RR, GPR, and PLS. Each has a "Help" button, and there is a "Parameters" button at the bottom.
- Non-linear approaches:** Contains an unchecked "Random Forest" checkbox with a "Help" button and a "Parameters" button.
- Deep learning approaches:** Contains checkboxes for "fully-connected NN" and "LSTM", each with a "Help" button, and a "CNN" checkbox with a "Parameters" button.

The Prediction Methods section provides access to a broad range of machine learning and statistical models used for predictive analysis. Users can select the desired prediction task and apply one or more algorithms depending on the data type and research objective.

Supported models include:

Linear methods:

LASSO – Least Absolute Shrinkage and Selection Operator,

RR – Ridge Regression,

ENR – Elastic Net Regression,

GPR – Gaussian Process Regression,

SVR – Support Vector Regression,

PLS – Partial Least Squares,

Non-linear methods:

RF – Random Forest,

Deep learning approaches:

fully connected NN – fully connected Neural Networks,

LSTM – Long Short-Term Memory,

CNN – Convolutional Neural Network

PRS analysis requires SNP positional information and produces downloadable CSV output files accompanied by relevant visualizations. For non-linear and deep learning models, users can further optimize performance by adjusting model-specific hyperparameters via the Parameters option.

Linear and statistical models – detail description

LASSO (Least Absolute Shrinkage and Selection Operator) regression applies L1 regularization to enhance model performance by performing both variable selection and regularization. In this method, the model is trained to predict exam scores using the LASSO and elastic net methods. Predictions are compared with actual exam grades using the formula:

$$\text{app.Pred_Val} = \text{XTest} * \text{coef} + \text{coef0};$$

RR (Ridge Regression) is a linear model incorporating L2 regularization to penalize large coefficients and mitigate multicollinearity. In the PANOMICS application, RR is implemented with a very small alpha parameter (e.g., 0.000001), approximating ridge regression. The model is trained using 5-fold cross-validation to find the optimal lambda value, enhancing stability and prediction accuracy.

ENR (Elastic Net Regularization) combines L1 and L2 penalties to leverage the strengths of both LASSO and Ridge Regression. In this approach, the alpha parameter is set to 0.75, balancing LASSO and Ridge contributions. The ENR model undergoes 5-fold cross-validation to determine the best regularization parameters, which are then used for predicting outcomes.

GPR– Fit a Gaussian process regression (GPR) model; GPR involves fitting a Gaussian Process Regression model using the `fitrgp` function. This model is trained on sample data, accommodating both univariate and multivariate outputs, and provides flexible prediction capabilities

`gprMdl = fitrgp(Tbl, ResponseVarName)` returns a Gaussian process regression (GPR) model trained using the sample data in Tbl, where ResponseVarName is the name of the response variable in Tbl.

(<https://www.mathworks.com/help/stats/fitrgp.html>)

SVR– Fit a support vector machine regression (SVR) model; SVR fits a support vector machine regression model to predictor data, utilizing kernel functions and soft-margin minimization techniques. The `fitrsvm` function supports various methods, including SMO and quadratic programming. For high-dimensional data, `fitrlinear` is used. For binary classification, `fitcsvm` and `fitclinear` are employed based on data dimensions

`fitrsvm` trains or cross-validates a support vector machine (SVM) regression model on a low- through moderate-dimensional predictor data set. `fitrsvm` supports mapping the predictor data using kernel functions, and supports SMO, ISDA, or L1 soft-margin minimization via quadratic programming for objective-function minimization.

To train a linear SVM regression model on a high-dimensional data set, that is, data sets that include many predictor variables, use `fitrlinear` instead.

To train an SVM model for binary classification, see `fitcsvm` for low- through moderate-dimensional predictor data sets, or `fitclinear` for high-dimensional data sets.

(<https://www.mathworks.com/help/stats/fitrsvm.html>)

PLS – Partial least-squares (PLS) regression – PLS regression calculates predictor and response loadings using the *plsregress* function, which identifies relationships between predictors and responses by specifying the number of components.

`plsregress(X,Y,ncomp)` returns the predictor and response loadings XL and YL, respectively, for a partial least-squares (PLS) regression of the responses in matrix Y on the predictors in matrix X, using ncomp PLS components.

(<https://www.mathworks.com/help/stats/plsregress.html>)

Non-linear methods – detail description

RF– Random forest implemented by **oobQuantilePredict** - Quantile predictions for out-of-bag observations from bag of regression trees

(<https://www.mathworks.com/help/stats/treebagger.oobquantilepredict.html>)

Deep Learning approaches:

Fully connected Neural Network (NN) – A feedforward neural network is used for non-sequential representation learning.

Architecture:

Feature input layer => Multiple fully connected layers => ReLU activation layers => Regression output layer

This architecture is optimized for general-purpose prediction tasks without explicit sequence modeling.

LSTM– LSTM networks are designed to model sequential dependencies in high-dimensional genomic data. SNPs are treated as ordered feature sequences, enabling the model to capture long-range interactions.

Architecture:

Sequence input layer => LSTM layer => Fully connected layer => Dropout layer => Fully connected output layer => Regression layer

Training is performed using the Adam optimizer with GPU acceleration when available. Mini-batch size, number of epochs, gradient threshold, and layer size are user-defined.

CNN – CNN models extract local patterns from genomic sequences using convolutional filters.

Architecture:

Sequence input layer => Stacked 1D convolutional layers (causal and same padding) => ReLU activation layers => Fully connected layers => Regression output layer

This architecture enables hierarchical feature extraction from SNP sequences and metabolomic profiles.

All deep learning and non-linear models support user-defined hyperparameter tuning via the Parameters button in the graphical interface, where users can modify the settings..

All deep learning models are trained using MATLAB's trainingOptions function with the Adam optimizer. GPU acceleration is automatically enabled if available; otherwise, training is performed on CPU. All prediction algorithms are accessible through a unified graphical interface. However, individual models employ different training, validation, and hyperparameter configuration procedures. These differences should be considered when comparing predictive performance across algorithms.

Model Training and Validation Strategies

The implemented algorithms use model-specific training and evaluation procedures.

LASSO, Ridge Regression, and Elastic Net Regression use a 70:30 training–testing split, with five-fold cross-validation performed within the training subset for regularization parameter selection.

Gaussian Process Regression, Support Vector Regression, and Partial Least Squares use five iterations of sequential index-based data partitioning.

Random Forest employs bootstrap aggregation and out-of-bag (OOB) prediction.

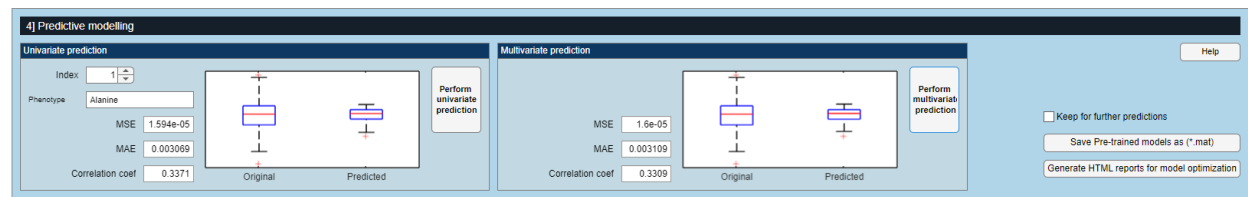
The deep-learning models (fullINN, LSTM, and CNN) use an 80:20 training–testing split, with the first 80% of observations used for training and the remaining 20% for testing.

Identical cross-validation folds are not shared across all algorithms, and systematic nested hyperparameter optimization is not implemented. Consequently, the reported results should be interpreted as exploratory comparisons under model-specific evaluation procedures.

4) Predictive Modelling

The Output and Predictive Modelling section manages the presentation, evaluation, and storage of prediction results. Outputs are provided in two formats: univariate (individual metabolites or target variables) and multivariate (complete metabolite or feature sets).

Model performance is assessed using standard evaluation metrics, including Mean Squared Error (MSE), Mean Absolute Error (MAE), and correlation coefficient (cc). In addition, predicted versus observed values can be visualized using boxplots for rapid assessment of model accuracy.



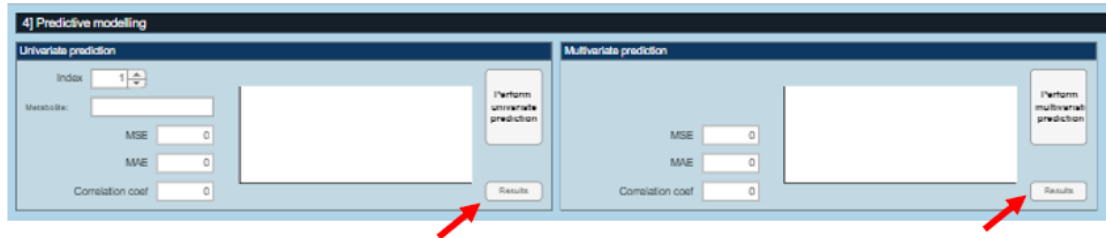
Results can be saved in *.mat* format within this section and later reloaded in the Input section for further analysis or integration with new datasets. To support model selection and benchmarking, the application also includes an option to generate HTML reports, enabling systematic comparison of prediction performance across all implemented methods.

Univariate Prediction Models				
Model	MSE	MAE	Correlation Coefficient	Evaluation
LASSO	0.276	0.428	0.000	✗
RR	0.203	0.353	0.548	✓
ENR	0.182	0.337	0.520	✓
GPR	0.188	0.347	0.434	✓
SVR	0.536	0.580	0.204	⚠
PLS	19948418914881752573211981119488.000	2973569181674939.500	0.008	✗
RF	0.192	0.353	0.418	✓
FullINN	0.273	0.425	-0.030	✗
CNN	0.160	0.338	0.275	⚠
LSTM	0.206	0.368	0.088	⚠
Multivariate Prediction Models				
Model	MSE	MAE	Correlation Coefficient	Evaluation
LASSO	0.257	0.400	0.261	✓
RR	0.200	0.365	0.241	✓
ENR	0.235	0.398	0.569	✓
GPR	0.004	0.051	0.018	✓
SVR	0.019	0.103	-0.052	⚠
PLS	9756606767019276585689359056896.000	1126998638812821.250	-0.004	⚠
RF	0.004	0.053	0.007	⚠
FullINN	0.009	0.073	-0.077	✗
CNN	0.004	0.052	-0.074	✗
LSTM	0.013	0.100	-0.078	✗

✓ Top third of models ⚠ Middle third ✗ Lowest performing models

5] Model Explainability and Diagnostics

After completing the prediction analysis, users can click the **“Results”** button to open a new window displaying detailed model evaluation, explainability, and diagnostic results.



The PANOMICS App includes a dedicated model explainability and diagnostic interface designed to support the interpretation and evaluation of machine-learning predictions. This interface provides access to feature importance analysis, SHAP-based interpretation, diagnostic visualizations, model-specific parameters, and prediction uncertainty assessments, depending on the selected algorithm.



Feature Importance Analysis

The application provides ranked feature importance tables and graphical representations of influential predictors. For LASSO, RR, and ENR, regression coefficients are used to examine predictor contributions. For nonlinear and deep-learning models, model-specific importance measures, including permutation-based approaches where implemented, provide complementary information about predictor relevance.

SHAP-Based Interpretation

SHAP (SHapley Additive exPlanations) analysis is available for supported models. The interface displays SHAP importance scores, mean SHAP values, and graphical representations of influential predictors.

Diagnostic Visualizations

The diagnostic panel includes residuals-versus-predicted plots, normal Q–Q plots, and observed-versus-predicted scatter plots. Learning curves are available for neural-network models when the corresponding training information has been recorded.

Regularization Paths and Model Parameters

For regularized regression methods, including LASSO, RR, and ENR, regularization paths illustrate changes in regression coefficients across different regularization strengths. Additional tables summarize selected model parameters and configuration settings.

Prediction Uncertainty and Error Assessment

For GPR, predictive standard deviations and corresponding prediction intervals are visualized.

For other supported models, residual-based error summaries include RMSE, MAE, mean residual, residual standard deviation, and maximum absolute prediction error.

Residual-based error summaries do not represent formal confidence intervals or prediction intervals.

These functionalities support the identification of influential molecular predictors, assessment of model behavior, and generation of biological hypotheses. However, model-derived feature importance does not establish biological causality.

6] Computational Benchmarking and Scalability

The computational requirements of PANOMICS depend on dataset dimensions, the selected machine-learning algorithm, and available hardware resources.

A theoretical computational and memory complexity analysis of all ten implemented algorithms is available in the accompanying GitHub repository:

<https://github.com/JanaSchwarzerova/PANOMICSApp/tree/main/docs/benchmarking>

The report (BENCHMARKING_THEORETICAL.md) evaluates theoretical scaling characteristics associated with training, validation, prediction, and interpretability procedures.

Illustrative dataset dimensions range from 239×249 to $10,000 \times 50,000$ (samples $\times$ features), considering desktop configurations with 8, 16, 32, and 64 GB RAM.

The analysis highlights algorithm-specific computational constraints, including quadratic memory scaling for exact GPR and increasing computational requirements for high-dimensional neural-network architectures.

Desktop and Web Implementations

Desktop performance depends on local CPU and memory resources. The web-based implementation is additionally constrained by server-side resource allocation and concurrent user demand.

No universal maximum input size can currently be guaranteed across all algorithms and deployment environments.

Empirical Benchmarking

A reproducible protocol has been established for future empirical evaluation of execution time, peak memory consumption, dataset dimensionality, and computational failures.

The available theoretical estimates do not represent experimentally measured runtime or peak memory consumption. Systematic empirical measurements and documentation updates are planned on a quarterly basis.

7| Limitations and Recommended Use

PANOMICS is primarily intended for exploratory genomic and multi-omics prediction using small- to medium-sized research datasets.

High-dimensional datasets with substantially more predictors than observations may increase the risk of overfitting. Model performance may also depend on sample size, predictor characteristics, hyperparameter configurations, and the selected validation strategy.

The implemented algorithms use different evaluation procedures, which should be considered when comparing predictive performance.

Feature importance and SHAP-based analyses support model interpretation but do not establish biological causality. Identified molecular predictors require subsequent biological validation.

The synthetic human dataset is intended as a methodological demonstration and does not establish clinical predictive utility.

Practical computational performance depends on the selected algorithm, dataset characteristics, available hardware, and deployment environment.